

OOPS with Java: Expected Questions with Programming Practice

UNIT 1: Java Basics, Program Structure, OOP Fundamentals, Packages

2 Marks Expected Questions

1. Define Java. Why is Java called platform independent?
2. What is bytecode in Java?
3. Differentiate between JVM and JRE.
4. What is the role of javac?
5. Define class and object.
6. What is a constructor?
7. What is method overloading?
8. What is method overriding?
9. What is encapsulation?
10. What is the use of the final keyword?
11. Define package in Java.
12. What is CLASSPATH?

5 Marks Expected Questions

1. Explain the features of Java: simple, secure, robust, portable, multithreaded and platform independent.
2. Explain JVM, JRE and JDK with a neat diagram.
3. Explain the structure of a Java program with an example.
4. Explain access modifiers in Java with a suitable table.
5. Differentiate between static and final members.
6. Explain constructor and its types with example.
7. Explain inheritance and its types in Java.
8. Differentiate between method overloading and method overriding.
9. Explain encapsulation with getter and setter methods.
10. Explain packages and their advantages.

8 Marks Expected Questions — Programming Based

1. Write a Java program to demonstrate class, object, constructor and method.

```
class Student {
    int rollNo;
    String name;

    Student(int r, String n) {
        rollNo = r;
        name = n;
    }

    void display() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(101, "Amit");
        s1.display();
    }
}
```

Expected theory part: explain class, object, constructor, instance variables and method.

2. Write a Java program to demonstrate method overloading.

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();

        System.out.println(c.add(10, 20));
        System.out.println(c.add(5.5, 4.5));
    }
}
```

```
        System.out.println(c.add(1, 2, 3));
    }
}
```

Expected theory part: explain compile-time polymorphism.

3. Write a Java program to demonstrate inheritance and method overriding.

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

Expected theory part: explain superclass, subclass, extends, overriding.

4. Write a Java program to demonstrate encapsulation.

```
class Employee {
    private int id;
    private String name;

    public void setId(int id) {
        this.id = id;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }
}
```

```

public class Main {
    public static void main(String[] args) {
        Employee e = new Employee();
        e.setId(101);
        e.setName("Rahul");

        System.out.println(e.getId());
        System.out.println(e.getName());
    }
}

```

Expected theory part: explain data hiding, private data members, getter/setter.

5. Write a Java program to demonstrate abstract class and interface.

```

abstract class Shape {
    abstract void draw();
}

interface Color {
    void fillColor();
}

class Circle extends Shape implements Color {
    void draw() {
        System.out.println("Drawing Circle");
    }

    public void fillColor() {
        System.out.println("Filling Red Color");
    }
}

public class Main {
    public static void main(String[] args) {
        Circle c = new Circle();
        c.draw();
        c.fillColor();
    }
}

```

Expected theory part: differentiate abstract class and interface.

UNIT 2: Exception Handling, File Handling, Multithreading

2 Marks Expected Questions

1. Define exception in Java.
2. Differentiate between error and exception.
3. What is the use of try and catch?
4. What is the use of finally block?
5. Differentiate between throw and throws.
6. What is checked exception?
7. What is unchecked exception?
8. Define user-defined exception.
9. What is multithreading?
10. What is the use of sleep() method?
11. What is synchronization?
12. What is FileInputStream?

5 Marks Expected Questions

1. Explain exception handling mechanism in Java.
2. Explain checked and unchecked exceptions with examples.
3. Explain try, catch, finally, throw, and throws.
4. Explain JVM reaction to exceptions.
5. Explain multiple catch blocks with example.
6. Explain user-defined exception with syntax.
7. Explain file reading and writing in Java.
8. Explain thread life cycle.
9. Explain thread creation using Thread class and Runnable interface.
10. Explain synchronization in Java.

8 Marks Expected Questions — Programming Based

1. Write a Java program to demonstrate exception handling using try-catch-finally.

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            int a = 10;  
            int b = 0;  
            int c = a / b;  
            System.out.println(c);  
        } catch (ArithmeticException e) {
```

```

        System.out.println("Cannot divide by zero");
    } finally {
        System.out.println("Finally block executed");
    }
}
}

```

Expected theory part: explain control flow of exception handling.

2. Write a Java program to demonstrate multiple catch blocks.

```

public class Main {
    public static void main(String[] args) {
        try {
            int arr[] = {10, 20, 30};
            System.out.println(arr[5]);
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic error");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array index error");
        } catch (Exception e) {
            System.out.println("General exception");
        }
    }
}

```

Expected theory part: explain specific-to-general catch ordering.

3. Write a Java program to create a user-defined exception.

```

class InvalidAgeException extends Exception {
    InvalidAgeException(String message) {
        super(message);
    }
}

public class Main {
    static void checkAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Not eligible to vote");
        } else {
            System.out.println("Eligible to vote");
        }
    }

    public static void main(String[] args) {
        try {
            checkAge(16);
        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

```
}  
}
```

Expected theory part: explain throw, throws, custom exception class.

4. Write a Java program to read data from a file.

```
import java.io.*;  
  
public class Main {  
    public static void main(String[] args) {  
        try {  
            FileInputStream fin = new FileInputStream("test.txt");  
            int ch;  
  
            while ((ch = fin.read()) != -1) {  
                System.out.print((char) ch);  
            }  
  
            fin.close();  
        } catch (IOException e) {  
            System.out.println("File error: " + e);  
        }  
    }  
}
```

Expected theory part: explain FileInputStream, read(), close().

5. Write a Java program to write data into a file.

```
import java.io.*;  
  
public class Main {  
    public static void main(String[] args) {  
        try {  
            FileOutputStream fout = new FileOutputStream("test.txt");  
            String data = "Welcome to Java File Handling";  
  
            byte[] b = data.getBytes();  
            fout.write(b);  
  
            fout.close();  
            System.out.println("Data written successfully");  
        } catch (IOException e) {  
            System.out.println("File error: " + e);  
        }  
    }  
}
```

Expected theory part: explain FileOutputStream and byte conversion.

6. Write a Java program to create threads using Thread class and Runnable interface.

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread using Thread class");
    }
}

class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread using Runnable interface");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();

        Thread t2 = new Thread(new MyRunnable());
        t2.start();
    }
}
```

Expected theory part: compare extending Thread and implementing Runnable.

7. Write a Java program to demonstrate synchronization.

```
class Table {
    synchronized void printTable(int n) {
        for (int i = 1; i <= 5; i++) {
            System.out.println(n * i);
        }
    }
}

class MyThread extends Thread {
    Table t;

    MyThread(Table t) {
        this.t = t;
    }

    public void run() {
        t.printTable(5);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
```

```
Table obj = new Table();

MyThread t1 = new MyThread(obj);
MyThread t2 = new MyThread(obj);

t1.start();
t2.start();
    }
}
```

Expected theory part: explain race condition and synchronized method.

UNIT 3: Java New Features

2 Marks Expected Questions

1. What is lambda expression?
2. Define functional interface.
3. What is Stream API?
4. What is the use of forEach()?
5. What is method reference?
6. What is default method in interface?
7. What is static method in interface?
8. What is diamond operator?
9. What is try-with-resources?
10. What is Java module?

5 Marks Expected Questions

1. Explain lambda expression with syntax and example.
2. Explain functional interface with example.
3. Explain Stream API operations: filter, map, sorted, collect.
4. Differentiate default and static methods in interface.
5. Explain method reference with example.
6. Explain try-with-resources.
7. Explain diamond operator with generics.
8. Explain Java module system.
9. Explain enhanced switch statement.
10. Explain annotations in Java.

8 Marks Expected Questions — Programming Based

1. Write a Java program to demonstrate lambda expression using functional interface.

```
interface Add {  
    int sum(int a, int b);  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Add obj = (a, b) -> a + b;  
        System.out.println(obj.sum(10, 20));  
    }  
}
```

Expected theory part: explain lambda syntax, functional interface, implicit return.

2. Write a Java program to demonstrate block lambda expression.

```
interface Add {
    int sum(int a, int b);
}

public class Main {
    public static void main(String[] args) {
        Add obj = (a, b) -> {
            return a + b;
        };

        System.out.println(obj.sum(10, 20));
    }
}
```

Expected theory part: explain why return is required in block lambda.

3. Write a Java program using Stream API to filter even numbers from a list.

```
import java.util.*;
import java.util.stream.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = Arrays.asList(10, 15, 20, 25, 30);

        nums.stream()
            .filter(n -> n % 2 == 0)
            .forEach(System.out::println);
    }
}
```

Expected theory part: explain stream, filter, lambda, terminal operation.

4. Write a Java program using Stream API to convert names into uppercase.

```
import java.util.*;
import java.util.stream.*;

public class Main {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("ram", "shyam", "mohan");

        names.stream()
            .map(String::toUpperCase)
            .forEach(System.out::println);
    }
}
```

Expected theory part: explain map() and method reference.

5. Write a Java program to demonstrate default and static methods in interface.

```
interface Vehicle {
    default void start() {
        System.out.println("Vehicle started");
    }

    static void service() {
        System.out.println("Vehicle service required");
    }
}

class Car implements Vehicle {
    public void drive() {
        System.out.println("Car is driving");
    }
}

public class Main {
    public static void main(String[] args) {
        Car c = new Car();
        c.start();
        c.drive();

        Vehicle.service();
    }
}
```

Expected theory part: default method can be inherited/overridden; static interface method is called using interface name.

6. Write a Java program to demonstrate try-with-resources.

```
import java.io.*;

public class Main {
    public static void main(String[] args) {
        try (FileWriter fw = new FileWriter("test.txt")) {
            fw.write("Java try-with-resources example");
            System.out.println("File written successfully");
        } catch (IOException e) {
            System.out.println(e);
        }
    }
}
```

Expected theory part: explain automatic resource closing.

7. Write a Java program to demonstrate diamond operator with ArrayList.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();

        list.add("Java");
        list.add("Python");

        for (String s : list) {
            System.out.println(s);
        }
    }
}
```

Expected theory part: explain generic type inference.

UNIT 4: Java Collections Framework

2 Marks Expected Questions

1. Define Collection Framework.
2. Differentiate between Collection and Collections.
3. What is List interface?
4. What is Set interface?
5. What is Map interface?
6. What is ArrayList?
7. What is LinkedList?
8. What is HashMap?
9. What is Iterator?
10. What is Comparable?
11. What is Comparator?
12. What is Hashtable?

5 Marks Expected Questions

1. Explain Java Collection Framework hierarchy.
2. Differentiate between List, Set and Map.
3. Explain ArrayList with methods.
4. Explain LinkedList and its working.
5. Explain Queue and Deque interfaces.
6. Explain HashMap and Hashtable.
7. Differentiate between ArrayList and Vector.
8. Explain Iterator and ListIterator.
9. Explain Comparable interface.
10. Explain Comparator interface.

8 Marks Expected Questions — Programming Based

1. Write a Java program to demonstrate ArrayList operations.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();

        names.add("Amit");
        names.add("Ravi");
        names.add("Neha");
```

```

        System.out.println("List: " + names);

        names.remove("Ravi");

        System.out.println("After removal: " + names);
        System.out.println("Size: " + names.size());
        System.out.println("Contains Amit: " + names.contains("Amit"));
    }
}

```

Expected theory part: explain dynamic array, ordered collection, duplicate support.

2. Write a Java program to demonstrate HashMap.

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>();

        map.put(101, "Amit");
        map.put(102, "Ravi");
        map.put(103, "Neha");

        for (Map.Entry<Integer, String> entry : map.entrySet()) {
            System.out.println(entry.getKey() + " " + entry.getValue());
        }
    }
}

```

Expected theory part: explain key-value pair, hashing, duplicate keys not allowed.

3. Write a Java program to sort employee objects using Comparable.

```

import java.util.*;

class Employee implements Comparable<Employee> {
    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int compareTo(Employee e) {
        return this.id - e.id;
    }
}

public class Main {

```

```

public static void main(String[] args) {
    ArrayList<Employee> list = new ArrayList<>();

    list.add(new Employee(103, "Ravi"));
    list.add(new Employee(101, "Amit"));
    list.add(new Employee(102, "Neha"));

    Collections.sort(list);

    for (Employee e : list) {
        System.out.println(e.id + " " + e.name);
    }
}

```

Expected theory part: Comparable gives natural sorting order using compareTo().

4. Write a Java program to sort employee objects using Comparator by name.

```

import java.util.*;

class Employee {
    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }
}

class NameComparator implements Comparator<Employee> {
    public int compare(Employee e1, Employee e2) {
        return e1.name.compareTo(e2.name);
    }
}

public class Main {
    public static void main(String[] args) {
        ArrayList<Employee> list = new ArrayList<>();

        list.add(new Employee(103, "Ravi"));
        list.add(new Employee(101, "Amit"));
        list.add(new Employee(102, "Neha"));

        Collections.sort(list, new NameComparator());

        for (Employee e : list) {
            System.out.println(e.id + " " + e.name);
        }
    }
}

```

```
    }  
}
```

Expected theory part: Comparator allows custom sorting criteria.

5. Write a Java program to demonstrate Stack.

```
import java.util.*;  
  
public class Main {  
    public static void main(String[] args) {  
        Stack<Integer> stack = new Stack<>();  
  
        stack.push(10);  
        stack.push(20);  
        stack.push(30);  
  
        System.out.println(stack.pop());  
        System.out.println(stack.peek());  
        System.out.println(stack);  
    }  
}
```

Expected theory part: explain LIFO principle.

6. Write a Java program to demonstrate Queue using PriorityQueue.

```
import java.util.*;  
  
public class Main {  
    public static void main(String[] args) {  
        Queue<Integer> q = new PriorityQueue<>();  
  
        q.add(30);  
        q.add(10);  
        q.add(20);  
  
        while (!q.isEmpty()) {  
            System.out.println(q.poll());  
        }  
    }  
}
```

Expected theory part: explain Queue, FIFO concept, PriorityQueue behavior.

UNIT 5: Spring Framework, Spring Boot, RESTful Web Services

2 Marks Expected Questions

1. What is Spring Framework?
2. What is dependency injection?
3. What is IoC container?
4. What is Spring Boot?
5. What is REST API?
6. What is @Controller?
7. What is @RestController?
8. What is @RequestMapping?
9. What is @GetMapping?
10. What is @PostMapping?
11. What is path variable?
12. What is request parameter?

5 Marks Expected Questions

1. Explain Spring Framework and its advantages.
2. Explain dependency injection with example.
3. Differentiate between Spring and Spring Boot.
4. Explain Spring Boot project structure.
5. Explain RESTful web services.
6. Explain HTTP methods GET, POST, PUT and DELETE.
7. Explain @RestController, @RequestMapping, @GetMapping, and @PostMapping.
8. Explain path variable and request parameter.
9. Explain request body in REST API.
10. Explain MVC architecture in Spring.

8 Marks Expected Questions — Programming Based

1. Write a Spring Boot REST controller to display a welcome message.

```
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController  
public class WelcomeController {
```

```
    @GetMapping("/welcome")
```

```
public String welcome() {  
    return "Welcome to Spring Boot";  
}  
}
```

Expected theory part: explain @RestController and @GetMapping.

2. Write a Spring Boot REST API using path variable.

```
import org.springframework.web.bind.annotation.*;  
  
@RestController  
public class StudentController {  
  
    @GetMapping("/student/{name}")  
    public String getStudent(@PathVariable String name) {  
        return "Student name is " + name;  
    }  
}
```

Expected theory part: explain path variable and URL mapping.

3. Write a Spring Boot REST API using request parameter.

```
import org.springframework.web.bind.annotation.*;  
  
@RestController  
public class CourseController {  
  
    @GetMapping("/course")  
    public String getCourse(@RequestParam String name) {  
        return "Course name is " + name;  
    }  
}
```

Example URL:

/course?name=Java

Expected theory part: differentiate path variable and request parameter.

4. Write a Spring Boot REST API to accept student data using request body.

```
import org.springframework.web.bind.annotation.*;

class Student {
    public int id;
    public String name;
}

@RestController
public class StudentController {

    @PostMapping("/student")
    public String addStudent(@RequestBody Student student) {
        return "Student added: " + student.id + " " + student.name;
    }
}
```

Expected theory part: explain @RequestBody and JSON-to-object mapping.

5. Write a simple REST controller demonstrating GET, POST, PUT and DELETE.

```
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api")
public class DemoController {

    @GetMapping("/item")
    public String getItem() {
        return "Item fetched";
    }

    @PostMapping("/item")
    public String createItem() {
        return "Item created";
    }

    @PutMapping("/item")
    public String updateItem() {
        return "Item updated";
    }

    @DeleteMapping("/item")
}
```

```
public String deleteItem() {  
    return "Item deleted";  
}  
}
```

Expected theory part: explain CRUD mapping with HTTP methods.

Most Important PYQ-Style 8-Markers to Prioritize

These are the questions most likely to become high-weightage exam questions because they combine theory + programming:

1. Write a Java program to demonstrate inheritance and method overriding.
2. Write a Java program to demonstrate encapsulation using getter and setter.
3. Write a Java program to demonstrate exception handling using try-catch-finally.
4. Write a Java program to create a user-defined exception.
5. Write a Java program to read and write data using FileInputStream/FileOutputStream.
6. Write a Java program to create threads using Thread class and Runnable interface.
7. Write a Java program to demonstrate synchronization.
8. Write a Java program using lambda expression and functional interface.
9. Write a Java program using Stream API to filter even numbers from a list.
10. Write a Java program using ArrayList and HashMap.
11. Write a Java program to sort objects using Comparable and Comparator.
12. Write a Spring Boot REST controller using GET, POST, path variable and request body.

This is the correct PYQ-style direction. Pure theory for 8 marks is weak for this subject; programming-based 8-mark questions are far more useful and exam-realistic.